

AI-Based Storage Tier Recommendation for Medical Image Repositories using Deep Learning Analytics

Research Gap Analysis and Literature Survey (Papers 1 to 5)

Course Instructor: Dr. Priya V | Submission: 30 July 2026

Name: Pratyush Chandrasekhar

Reg No: 24BIT0226

1. What This Project Is About

Hospitals store a very large number of scans such as CT, MRI and X-ray. Keeping all of them on fast storage is expensive. Moving all of them to cheap storage is also a problem, because cheap storage can take hours to give a file back, and a doctor cannot wait that long.

Amazon S3 offers different storage classes. S3 Standard is fast but costly. Glacier Deep Archive is very cheap but slow. The aim of this project is to use deep learning to decide which storage class each scan should go into.

For this, I studied five research papers. For each paper I have written what the authors did, what data they used, what is good about it, what is missing, and what gap is left that our project can fill.

2. The Five Papers I Studied

#	Title	Journal	Year
P1	Cloud storage tier optimization through storage object classification	Computing (Springer)	2024
P2	Recent Advances in Big Medical Image Data Analysis Through Deep Learning and Cloud Computing	Electronics (MDPI)	2024
P3	Using Compressed JPEG and JPEG2000 Medical Images in Deep Learning: A Review	Applied Sciences (MDPI)	2024
P4	Deep learning based medical image compression using cross attention learning and wavelet transform	Scientific Reports (Nature)	2025
P5	A Recommendation System Based on AI for Storing Block Data in the Electronic Health Repository	Frontiers in Public Health	2022

3. Research Gap Analysis (Paper by Paper)

3.1 Paper 1 — Cloud storage tier optimization through storage object classification

A. Q. Khan, M. Matskin, R. Prodan, C. Bussler, D. Roman and A. Soylu, *Computing*, vol. 106, pp. 3389-3418, 2024.

In one line: This paper sorts cloud files into four storage tiers using simple rules, but it never looks inside the file.

What the authors did: The authors sort stored files into four tiers: premium, hot, cold and archive. Each file is given a score based on three things , its size, its age, and how often it is opened. If the score crosses a fixed limit, the file goes into that tier. For example, a score above 0.7 means the file goes to

the hot tier. They also try a second method using game theory, where the score weights adjust themselves instead of being fixed by hand.

Data they used: They did not have real company data. So they built their own test data from a public access log on Kaggle. It covers 871 days, 14,321 files and about 2.9 million read requests. File sizes were missing from the log, so they added random sizes and made four versions, from 1 TB up to 67 TB of total data. They used Google Cloud prices to calculate the cost.

Advantages:

- Uses four tiers instead of the usual two. This matches Amazon S3 well, because S3 also has four main classes.
- The method is simple and fast to run. It does not need a big training system, so it can be run again and again on many files.
- It works on any cloud platform and in any industry. The authors mention healthcare as one possible area.
- The game theory version gave a slightly lower cost and also ran faster.

Limitations:

- It was tested only on made-up data. The authors themselves say that testing on real data is still pending.
- The cost calculation leaves out the fee charged for moving a file between tiers, and the penalty for deleting a file too early. So the real savings will be lower than what they report.
- It only saves money if files are kept for about 90 days or more.
- The score limits, such as 0.7, were chosen by hand. If the usage pattern changes, someone has to set them again manually.
- It only reads file details. It never opens the file or looks at what is inside it.

RESEARCH GAP

This method only looks at file details such as size, age and how many times the file was opened. It never looks inside the image. In a hospital this is a problem. Two scans can have the same size, the same age and the same number of views, but one may show a normal result and the other may show a problem that the doctor is still following up. This method would treat both scans in exactly the same way. So the important scan may get sent to archive storage, where it takes hours to bring back. The paper also has no way to keep a file in storage for the number of years that hospital rules require.

How we can improve on this: We can add one more input to the score — a value that comes from looking at the image itself using a deep learning model. Along with this we can also use DICOM details like the scan type and the body part. The four-tier idea is good and we will keep it, but instead of setting the limits by hand we will learn them from the hospital's own record of which scans were opened again. We will also add the S3 transfer charges that this paper left out.

3.2 Paper 2 — Recent Advances in Big Medical Image Data Analysis Through Deep Learning and Cloud Computing

M. Y. Shakor and M. I. Khaleel, Electronics, vol. 13, no. 24, art. 4860, 2024.

In one line: This is a survey paper. It proves hospital storage is expensive and that deep learning reads scans well, but it never joins the two ideas.

What the authors did: This is a review paper, not an experiment. The authors searched IEEE Xplore, PubMed, Scopus, Web of Science and Google Scholar for work published between 2019 and November

2024. They picked 37 papers and studied them. They also collected results from real hospitals that moved to the cloud, and used all of this to suggest a step-by-step plan for hospitals.

Data they used: There is no experiment and no dataset of their own. The evidence is the 37 papers plus real hospital cases. One example is a group of five hospitals handling 1.2 million scans a year on a hybrid cloud. Another is eight cancer centres sharing 15,000 anonymous cases.

Advantages:

- It gives clear numbers that show why the cloud helps. Time to open a scan dropped from 5 to 7 minutes down to about 30 seconds. Infrastructure cost dropped by about 40 percent. Processing time dropped by about 60 percent.
- It clearly says that storing and moving hospital data is one of the biggest cost problems. This is exactly the problem our project is trying to solve.
- It supports the design choices we need to justify. Hybrid cloud improved rule compliance by about 50 percent. Federated learning kept about 95 percent of the accuracy. Edge computing cut delay by about 40 percent.
- It is honest about failures. 35 percent of hospitals faced data joining problems and 5 percent had a security incident.

Limitations:

- It is only a review. There is no algorithm, no experiment and no code that we can run or repeat.
- Many of the percentages come from hospital reports. The paper does not explain how those numbers were measured, so we cannot check them.
- Storage is discussed only as a general cost problem. The paper never goes down to the level of deciding where one particular scan should be kept.
- Deep learning is used only to read scans and find disease. It is never used to manage the storage system itself.

RESEARCH GAP

This paper proves two facts. First, storing hospital scans costs a lot of money. Second, deep learning is already good enough to read those scans. But the paper keeps these two facts separate. In every study it reviews, deep learning is used only on the doctor's side, to find disease. It is never used on the storage side, to decide where a file should be kept. Nobody takes the result of the image model and sends it back to the storage system. That is the empty space our project fills. The hospital already runs the scan through a model once. That same result can also tell us where the scan should be stored.

How we can improve on this: We will design our system so that one pass of the model does two jobs at the same time — helping the doctor, and deciding the storage tier. We will also measure our project the way this paper shows hospital staff actually judge a system: how fast a scan comes back, how much the monthly bill is, and whether the rules are followed. Accuracy alone is not enough. We will also use the hybrid cloud design this paper recommends, so that patient details stay protected while S3 handles the bulk image storage.

3.3 Paper 3 — Using Compressed JPEG and JPEG2000 Medical Images in Deep Learning: A Review

I. A. Urbaniak, Applied Sciences, vol. 14, no. 22, art. 10524, 2024.

In one line: This paper studies how much a scan can be compressed before deep learning starts making mistakes, but it never links compression to storage tier.

What the authors did: This is a review of what happens to deep learning models when medical images are compressed in a lossy way, which means some detail is thrown away for ever. The author looks only at JPEG and JPEG2000, because these are the only two lossy formats that the DICOM medical standard allows. She compares studies where models were trained and tested on compressed, uncompressed and mixed images.

Data they used: No new experiments were done. The paper collects results from earlier studies. Some used ordinary image sets like MNIST and CIFAR-10. Others used real medical data such as mammograms and digital pathology slides.

Advantages:

- It gives real numbers for the trade-off between saving space and losing accuracy. Pathology images were compressed by 85 percent and the model still kept about 95 percent of its performance.
- It shows where the safe limit ends. For mammograms, accuracy only started dropping after compression crossed about 5000 to 1.
- It gives a useful rule: if a model is trained on images with a mix of different compression levels, it works better on all of them.
- It clearly lists lower storage cost and faster training as benefits, and sticks only to formats that DICOM allows.

Limitations:

- The studies do not agree with each other. Some say accuracy drops only at very high compression. Others say accuracy went up, because the model learned from the compression marks themselves.
- No safe compression limit has been fixed for each scan type or body part. Every result is tied to one particular dataset.
- The author states directly that there is not enough evidence yet to reach a common conclusion.
- There is a risk that a model learns the compression marks instead of the actual disease. It then scores well in training but fails on new images.
- Compression is treated as a single setting applied to the whole dataset. It is not changed from one scan to another.

RESEARCH GAP

In this paper, compression is decided once for the entire archive and never changed. It is also never connected to storage tier. But these are really the same decision seen from two sides. A scan that will almost certainly never be opened again can be compressed heavily to save space. A scan that a doctor is still using must be kept at full quality. No study here changes the compression level for each scan based on how likely that scan is to be needed again, and no study joins the compression choice and the storage class choice into one decision.

How we can improve on this: In our project, compression will be a second output of the same system instead of a fixed setting. If a scan is sent to a hot tier, we keep it at full quality. If it goes to deep archive, we compress it using a ratio that this paper has already shown to be safe for that scan type. This also avoids the problem of the model learning compression marks, because training will still use the full quality copies from the hot tier.

3.4 Paper 4 — Deep learning based medical image compression using cross attention learning and wavelet transform

F. Dai, Scientific Reports, vol. 15, art. 40008, 2025.

In one line: This paper builds a deep learning model that finds the important parts of a scan, but it uses that information only to compress, and then throws it away.

What the authors did: The author builds a compression system with three parts. First, a wavelet transform breaks the image into different levels of detail. Second, a cross attention module decides which parts of the image carry the most information and protects those parts. Third, a small autoencoder cleans up the result before it is finally compressed. A decoder then rebuilds the image.

Data they used: Three public CT datasets were used: LIDC-IDRI, LUNA16 and MosMed. Each was split into 70 percent training, 15 percent validation and 15 percent testing, with 5-fold cross validation. Brain and head MRI scans were also used. Average results were PSNR 40.43 dB, SSIM 0.9715 and 635 milliseconds per image. Training took about 60 to 90 minutes per dataset on an Intel Core i5 CPU.

Advantages:

- It beats JPEG2000, HEVC and BPG, and also three recent deep learning methods, on all quality measures at the same time. So it saves space without losing image quality.
- The attention part looks at the content of the image. It protects the areas that matter medically instead of treating the whole image equally. This is exactly the ability our project needs.
- It was trained on a normal CPU, not an expensive GPU. So it is realistic for a student project or a hospital with a small budget.
- The author clearly says the work is meant for telemedicine and cloud based healthcare storage.

Limitations:

- It was tested mostly on CT and brain MRI. Other scan types are listed as future work.
- It takes about 635 milliseconds per image. This is fine for background work but slow if a hospital is uploading many large studies at once.
- Quality is measured only by mathematical scores such as PSNR and SSIM. No radiologist checked whether the compressed scans are still good enough to make a diagnosis.
- It says nothing about storage. It decides how small to make the file, but not where that file should be kept. There is no cloud cost calculation anywhere in the paper.

RESEARCH GAP

This paper creates exactly the information our project needs, and then throws it away. The attention module already works out which parts of a scan carry important medical detail. But this information is used only to decide how many bits to spend on each area. After compression it is deleted. It is never passed on to any other part of the system. The same compression level is also used for every scan, whether that scan will be opened tomorrow or never again. The paper reduces the size of one image but never looks at the hospital's total bill, which depends on storage class, how long files are kept, and how often they are pulled back.

How we can improve on this: Instead of throwing away the attention result, we will feed it into our tier decision. If a scan gets a high attention score, it has a lot of medical detail, so we keep it at good quality in a faster tier. If the score is low, we can compress it more and move it to archive. We will also test our system in a better way than this paper does: not only PSNR and SSIM, but also whether a disease detection model still works on the compressed scan, and what the monthly S3 bill actually comes to.

3.5 Paper 5 — A Recommendation System Based on AI for Storing Block Data in the Electronic Health Repository

V. Mani, C. Kavitha, S. S. Band, A. Mosavi, P. Hollins and S. Palanisamy, Frontiers in Public Health, vol. 9, art. 831404, 2022.

In one line: This is the closest paper to our idea, but it works on text records instead of images, and it decides only once with no follow up.

What the authors did: The authors build a system that recommends where a block of health data should be stored. They describe the data using four things: how sensitive it is, how large it is, the type of medical care, and the patient's background. They then rate each storage option on five points: storage, cost, security, privacy and performance. Experts fill in a questionnaire, and this is used to train classifiers. They compared Multilayer Perceptron, Random Forest, Random Tree and IB1 using the WEKA tool, along with a deep learning model.

Data they used: The data was not taken from a real hospital system. They created it themselves from a small number of expert ratings, giving two sets of 1,200 and 2,000 records. Random Forest gave the best result, 98.21 percent accuracy on the 1,200 record set. The deep learning model gave 88.70 percent. One point to note is that the conclusion of the paper says the highest accuracy was 96.4 percent, which does not match the 98.21 percent given in the results section.

Advantages:

- This is the closest published work to our project. It shows that choosing a storage location can be treated as a machine learning problem.
- It judges storage options on five different points, not only on cost. This is closer to how a real hospital decides.
- Privacy, security and legal rules are built into the decision from the start.
- It reports precision, recall and F1 score, not only accuracy, so the results can be judged properly.

Limitations:

- The training data comes from expert opinion, not from real usage. So the model learns what experts say they would do, not what actually happened.
- Accuracy dropped on the 2,000 record set because the data was unbalanced. The model also performed poorly on the 'not allowed' class.
- Once a record is labelled, the label is never changed again. So the system cannot adjust when usage patterns change over time.
- The choice is only between three broad options. It never deals with the different tiers inside one cloud provider, which is what our project needs.
- It works on text records and data blocks. Medical images, which take up most of the storage, are never handled, and no image is ever examined.
- It was published in 2022, which is outside the 2023 to 2026 range preferred in the guidelines. I have still included it because it is the closest work to our idea.

RESEARCH GAP

This paper is the closest to our project, and it stops exactly where our work needs to start. The decision is taken only once, when the data first arrives, and it is based on expert opinion. After that the decision is never checked again. If a file turns out to be opened far more often than expected, nothing happens. The system also works on text records, not images, and it only reads details about the data. It never looks at the actual content. In an image archive this means the most useful piece of information available is completely unused.

How we can improve on this: We will keep the good idea of judging storage on several points, but we will make three changes. First, instead of expert opinion we will train on the hospital's real record of which scans were opened again, so the model learns actual behaviour. Second, we will work on one scan at a time instead of a block of text data, and we will add features taken from the image itself. Third, we will check the decision again from time to time using S3 Lifecycle rules, so a scan can be moved up or down as its usage changes. We will also treat a wrong move to archive as a more serious

mistake than paying slightly extra, because a doctor waiting hours for a scan is worse than a small increase in the bill.

4. Overall Research Gap

Putting all five papers together, this is what I found:

- Paper 1 can sort files into storage tiers, but it only reads file details and never looks inside the file.
- Paper 2 proves that hospital storage is costly and that deep learning reads scans well, but it never uses deep learning to manage storage.
- Paper 3 shows how much an image can be compressed safely, but treats compression as one fixed setting and never links it to storage tier.
- Paper 4 builds a model that finds the important parts of a scan, but uses that only for compression and then deletes it.
- Paper 5 does treat storage choice as a learning problem, but works on text records, learns from expert opinion instead of real usage, and decides only once.

So the gap is this. No existing work uses deep learning on the medical image itself, together with the record of how often that scan was opened before, to predict whether it will be needed again and to place it in the correct Amazon S3 storage class. That is what our project will do.

5. Literature Survey Table

Paper	Method	Dataset	Advantages	Limitations	Research Gap
P1 — Khan et al. (2024), Computing, Springer	Gives each file a score from its size, age and how often it is opened, then sends it to one of four tiers: premium, hot, cold or archive. A second version uses game theory so the score weights adjust on their own.	Made-up data built from a Kaggle access log: 871 days, 14,321 files, 2.9 million read requests. Four size versions from 1 TB to 67 TB. Google Cloud prices used.	Four tiers match Amazon S3 well. Simple and fast to run. Works on any cloud. Game theory version was cheaper and faster.	Tested only on made-up data. Leaves out file transfer fees and early delete penalties. Needs files kept 90+ days. Score limits set by hand. Never looks inside the file.	Two scans with the same size, age and view count are treated the same, even if one is medically important. No rule to keep files for the legally required number of years.
P2 — Shakor & Khaleel (2024), Electronics, MDPI	A review of 37 papers published between 2019 and Nov 2024, taken from IEEE Xplore, PubMed, Scopus, Web of Science and Google Scholar, plus real hospital cases, turned into a step-by-step plan for hospitals.	No dataset of their own. 37 papers plus hospital cases, such as five hospitals handling 1.2 million scans a year and eight cancer centres sharing 15,000 cases.	Scan opening time fell from 5-7 min to about 30 sec. Cost fell about 40%. Processing time fell about 60%. Hybrid cloud improved rule compliance by 50%.	Only a review, no experiment or code. Hospital numbers cannot be checked. Storage covered only in general. Deep learning used only to find disease, never to manage storage.	Proves storage is costly and deep learning reads scans well, but never joins the two. The output of the image model is never sent back to the storage system.
P3 — Urbaniak (2024), Applied Sciences, MDPI	A review of what happens to deep learning models when medical images are compressed in a lossy way, covering only JPEG and JPEG2000 because those are the two formats DICOM allows.	No new experiments. Collects results from earlier studies using MNIST, CIFAR-10, mammograms and digital pathology slides.	Gives real numbers: pathology images compressed 85% still kept about 95% of performance; mammogram accuracy dropped only past 5000:1. Mixed compression levels in training help.	Studies disagree with each other. No safe limit fixed per scan type or body part. Author says evidence is not enough yet. Models may learn compression marks instead of disease.	Compression is one fixed setting for the whole archive. It is never changed per scan and never connected to which storage tier the scan goes into.
P4 — Dai (2025), Scientific Reports, Nature	Wavelet transform splits the image into detail levels, a cross attention module protects the important areas, a small autoencoder cleans it up, then the image is compressed and rebuilt.	LIDC-IDRI, LUNA16 and MosMed CT datasets, split 70/15/15 with 5-fold cross validation, plus brain and head MRI. Avg PSNR 40.43 dB, SSIM 0.9715, 635 ms per image. Trained on an i5 CPU.	Beats JPEG2000, HEVC, BPG and three recent deep learning methods on all quality measures at once. Attention protects medically important areas. Runs on a normal CPU.	Mostly CT and brain MRI only. 635 ms per image is slow for bulk upload. Judged only by maths scores, no doctor checked the images. Says nothing about storage or cost.	The attention module finds which parts of a scan matter, then deletes that information after compressing. It is never reused. Same compression for every scan. No total cost view.
P5 — Mani et al. (2022), Frontiers in Public Health	Describes health data by sensitivity, size, care type and patient background, rates storage options on storage, cost, security, privacy and performance, then trains MLP, Random Forest, Random Tree and IB1 in WEKA plus a deep learning model.	Made-up sets of 1,200 and 2,000 records built from a small number of expert ratings. Random Forest best at 98.21% on 1,200 records; the conclusion says 96.4%. Deep learning model 88.70%.	Closest work to our project. Shows storage choice can be a machine learning problem. Judges storage on five points, not just cost. Privacy and legal rules built in.	Trained on expert opinion, not real usage. Accuracy dropped on the unbalanced 2,000 record set. Labels never updated. Only three broad options, not tiers. Text records only, no images.	Decides only once when data arrives and never checks again. Works on text blocks, not scans. Reads only data details, never the actual image content.

AI-Based Storage Tier Recommendation for Medical Image Repositories using Deep Learning Analytics

Research Gap Analysis and Literature Survey (Papers 6 to 10)

Course Instructor: Dr. Priya V | Submission: 30 July 2026

Name: Subhrojyoti Das

Reg No: 24BIT0194

1. What This Project Is About

Hospitals store a very large number of scans such as CT, MRI and X-ray. Keeping all of them on fast storage is expensive. Moving all of them to cheap storage is also a problem, because cheap storage can take hours to give a file back, and a doctor cannot wait that long.

Amazon S3 offers different storage classes. S3 Standard is fast but costly. Glacier Deep Archive is very cheap but slow. The aim of this project is to use deep learning to decide which storage class each scan should go into.

My teammates have already covered ten papers between them. To avoid repeating their work, I studied five different papers that no one else in the team has taken. My five papers approach the same problem from the storage system side and the hospital infrastructure side. For each paper I have written what the authors did, what data they used, what is good about it, what is missing, and what gap is left that our project can fill.

2. The Five Papers I Studied

#	Title	Source / Publisher	Year
P6	Harmonia: Enhancing Data Placement and Migration in Hybrid Storage Systems via Multi-Agent Reinforcement Learning	arXiv preprint (SAFARI Group, ETH Zurich)	2025 (rev. 2026)
P7	Machine Learning-Driven Intelligent Memory System Design: From On-Chip Caches to Storage	arXiv preprint (ETH Zurich)	2026
P8	Cold-RL: Learning Cache Eviction with Offline Reinforcement Learning for NGINX	arXiv preprint (open-source code)	2025
P9	Kaapana: A Comprehensive Open-Source Platform for Integrating AI in Medical Imaging Research Environments	arXiv preprint (DKFZ, Germany)	2025
P10	Project Imaging-X: A Survey of 1000+ Open-Access Medical Imaging Datasets for Foundation Model Development	arXiv preprint (127 authors)	2026

A note on why all five are preprints

My teammates' papers are from journals such as Springer, MDPI and Nature. All five of my papers are arXiv preprints instead. This is not an accident, and it is worth explaining.

- The guideline asks for recent work (2025 to 2026). Journal review takes 12 to 24 months, so almost nothing published in a journal today is actually from 2025 or 2026 in terms of the research itself.
- Papers P6 and P7 come from the SAFARI research group at ETH Zurich, led by Prof. Onur Mutlu. This is one of the best known computer architecture groups in the world and their work reliably appears later at top conferences such as ISCA, MICRO and HPCA. Paper P9 comes from DKFZ, the German Cancer Research Centre.
- All five are free to download with no paywall and no login, which matters for a student project.

- Before final submission I will search each title again and cite the journal or conference version if one has appeared by then.

3. Research Gap Analysis (Paper by Paper)

3.1 Paper 6 | Harmonia: Enhancing Data Placement and Migration in Hybrid Storage Systems via Multi-Agent Reinforcement Learning

R. Nadig, V. Arulchelvan, R. Bera, T. Shahroodi, G. Singh, A. Kakolyris, I. E. Yuksel, M. Sadrosadati, J. Park and O. Mutlu, arXiv:2503.20507, 2025 (revised 2026).

In one line: *Two AI agents work together to decide where a file should be kept and when it should be moved, but they only watch how the file behaves and never look at what is inside it.*

What the authors did:

Earlier systems used one AI agent to do one job. Harmonia uses two small reinforcement learning agents that talk to each other. The first agent is the placement agent. When a new file arrives it decides which device that file should be written to. The second agent is the migration agent. It watches files that are already stored and decides when a file should be moved to a different device. Because the two agents share what they have learned, they do not fight each other. Both agents are deliberately kept small so that the decision can be made faster than the storage device itself can respond.

Data they used:

Twenty-five real workloads were used. These include SYSTOR17 enterprise office traces (LUN0 to LUN4), RocksDB database traces (ssd-00 to ssd-04), YCSB cloud database traces (YCSB-B to YCSB-F), MLPerf Storage machine learning traces (ResNet50 and CosmoFlow), three traces from large language model work, and five production traces from Meta called Baleen which were kept aside as unseen tests. Testing was done on real hardware: an Intel Optane P4800X SSD, an Intel D3-S4510 SSD, a Seagate ST1000DM010 hard disk and an ADATA SU630 SSD, arranged in two, three and four device combinations.

Advantages:

- It handles both jobs together. Placement decides where a file goes at the start, migration decides when to move it later. Our project needs both, because a scan that is busy today may be forgotten next year.
- The improvement is large. Compared with the previous best method it reduced average waiting time by 29.3 percent on a speed-focused setup and 44.8 percent on a cost-focused setup, and it gave 136.7 percent more operations per second on the cost-focused setup.
- It works with three and four devices, not just two. This matters because Amazon S3 also has four main classes.
- It is extremely light. A decision takes about 240 nanoseconds and both agents together need only 206 KiB of memory. This proves that a learning based tier system does not slow the storage down.
- It was tested on unseen workloads from Meta that it had never trained on, so the results are not just memorised.

Limitations:

- There is nothing medical anywhere in the paper. The workloads are office desktops, databases and machine learning jobs. No scan, no hospital, no patient.
- The agents only see numbers about behaviour: how big the request is, whether it is a read or a write, how often it is accessed. They never see what the file contains.
- The word "cost" in this paper means cheaper hardware, not a cloud bill. There is no rupee or dollar figure anywhere, and no charge for moving a file or pulling it back.

- The settings of the agents were tuned once, offline, by hand. They do not retune themselves.
- There is no idea of a file that must legally be kept for a fixed number of years.

RESEARCH GAP

Harmonia solves the placement and migration problem beautifully, but it is completely blind to meaning. Its agents judge a file only by how it has behaved so far. In a hospital this fails in a specific way. A follow-up CT of a cancer patient and a routine chest X-ray of a healthy person can have exactly the same size, the same age and the same number of past opens. Harmonia would treat them identically. The cancer scan could be sent to deep archive, and when the doctor asks for it at the next appointment it would take hours to come back. The paper also measures cost as cheaper hardware, not as the actual monthly cloud bill, and it has no way to obey the rule that a hospital must keep a scan for a fixed number of years.

How we can improve on this:

We will keep the two-agent design because it fits our problem exactly, and we will make three changes. First, we will add new inputs to what the agents can see: features taken from the image itself using a deep learning model, plus DICOM details such as the scan type, the body part and the patient age. Second, we will replace the reward. Instead of rewarding only speed, we will reward the actual Amazon S3 monthly cost, including the transfer and retrieval charges this paper leaves out. Third, we will make the penalty unequal on purpose. Paying a little extra money is a small mistake; sending a scan a doctor needs into deep archive is a big mistake, and the reward will say so.

3.2 Paper 7 | Machine Learning-Driven Intelligent Memory System Design: From On-Chip Caches to Storage

R. Bera, R. Nadig and O. Mutlu, *arXiv:2603.14583*, 2026.

In one line: *This paper shows that a learning system beats hand-written rules at deciding where data should be kept, but the system it describes looks at only six numbers, and not one of them says what the data actually is.*

What the authors did:

This is an overview paper. It brings together three learning based methods built by the same group. Pythia is a reinforcement learning prefetcher for processor caches. Hermes is a small perceptron that predicts whether data will be found in the cache or not. Sibyl is the one that matters to us: it is a reinforcement learning system that decides which storage device a page of data should be placed on in a mixed storage system. The paper argues that memory and storage systems should tune themselves instead of relying on rules written by engineers.

Data they used:

This being an overview paper, there are no new experiments. Sibyl was originally tested on real machines using fourteen storage traces from Microsoft Research Cambridge, collected from enterprise servers. Sibyl describes each page using exactly six numbers: request size, request type (read or write), access interval, access count, how much free space is left on the fast device, and where the page is currently placed.

Advantages:

- It proves the core idea of our project is sound. Sibyl beat hand-written rules by 19.9 to 45.9 percent and beat earlier learning based methods by 34.1 to 68.8 percent.
- It was tested on real hardware inside the operating system storage layer, not only in a simulator.
- It states its six inputs openly. This is very useful for us, because we can point at that exact list and show what is missing from it.

- It is honest about what Sibyl cannot do. The authors say plainly that Sibyl handles placement but not migration, which is why they later built Harmonia.
- It gives us a clear story for the report: across the whole memory hierarchy, learning beats hand-written rules again and again.

Limitations:

- All six inputs describe system behaviour. Not one describes the content or the purpose of the data.
- The Microsoft Research Cambridge traces come from ordinary enterprise servers. They contain no medical data of any kind.
- There is a cold start problem. A file that has just arrived has no access history, so four of the six inputs are effectively empty and the system is guessing.
- Sibyl decides between storage devices inside one machine. It does not deal with cloud storage classes or cloud pricing.
- Being an overview paper, it presents no new experiment that we can repeat.

RESEARCH GAP

The six-number input list of Sibyl is the research gap of our whole project stated in a single line. Request size, read or write, access interval, access count, free space and current location. Every one of these describes behaviour. Not one describes what the data is. A hospital, however, already knows a great deal about a scan the moment it is created: the modality, the body part, the patient age, whether it is urgent, and what the report says. All of this is free and already stored in the DICOM header, and all of it is thrown away by this approach. The cold start problem makes it worse. A newly created scan has no access history at all, so exactly when the placement decision must be made, the system knows the least.

How we can improve on this:

We will extend the six-number list with two new kinds of input. The first is DICOM metadata, which is already present in every scan and costs nothing to read. The second is a small feature vector produced by a deep learning model that looks at the image itself. This directly fixes the cold start problem, because a brand new scan has no access history but it does have a modality, a body part and an image. Our system will therefore be able to place a scan correctly on the very first day, which Sibyl cannot do.

3.3 Paper 8 | Cold-RL: Learning Cache Eviction with Offline Reinforcement Learning for NGINX

A. Gupta and A. Bhayani, *arXiv:2508.12485*, 2025. Source code released on GitHub under CC-BY 4.0.

In one line: *This paper trains its AI entirely from old log files instead of experimenting on a live system, which is exactly the only way a hospital would ever allow us to train ours.*

What the authors did:

NGINX is a very widely used web server. When its cache becomes full it throws out the least recently used file. The authors replaced that fixed rule with a small neural network called a dueling Deep Q-Network. When space is needed, the network looks at the K least recently used objects and scores each one using four things: how old it is, how big it is, how many times it has been opened, and how long it is allowed to live. It then picks which ones to remove. The whole decision must finish in microseconds.

Data they used:

No live system was ever experimented on. Real NGINX access logs were replayed through a cache simulator, and the model learned from that replay. The reward was simple: if a file that was kept is opened again before it expires, the model earns a point. Three cache sizes were tested, 25 MB, 100 MB and 400 MB.

Advantages:

- The training method is the most useful part for us. Learning happens offline from recorded logs. We can never let a learning agent experiment freely on a real hospital archive, so this is the only workable route.
- The four inputs it uses (age, size, number of opens, time to live) map almost one to one onto information we already have for every scan in a hospital archive.
- The improvement is real. At a 25 MB cache the hit rate rose from 14.36 percent to 35.38 percent, and at 100 MB from 75.30 percent to 86.75 percent.
- The overhead is under 2 percent of CPU, so it does not slow the server down.
- It is honest. At a 400 MB cache the learned method only matched the old rule at about 91.8 percent and gave no benefit. The authors report this instead of hiding it.
- The code is public, so we can actually study a working implementation.

Limitations:

- It is a web cache, not a storage archive. It decides what to delete, not which of four storage classes a file should live in.
- There are only two outcomes: stay in the cache, or be removed. Our project needs four.
- The reward is a simple yes or no. It has no idea that some mistakes are far more costly than others.
- There is no money in the paper at all. Nothing is measured in cost.
- The benefit shrinks as the cache grows. When there is enough space, the learned method stops helping.
- There is no medical data and no image of any kind.

RESEARCH GAP

Cold-RL solves a two-way question: keep this file, or throw it away. Our project has to solve a four-way question across S3 Standard, Infrequent Access, Glacier and Deep Archive. More importantly, the cost of being wrong is completely different. If Cold-RL wrongly throws out a web page, the server fetches it again in a few milliseconds and nobody notices. If our system wrongly sends a scan to Glacier Deep Archive, a doctor may wait twelve hours for an image they need now. A simple yes-or-no reward, which is what this paper uses, cannot express that difference at all. It treats every mistake as equally bad.

How we can improve on this:

We will copy the offline training pipeline almost exactly, because it is the only approach a hospital would permit: take recorded access history, replay it through a simulator, and train from that replay. Then we will make two changes. We will widen the decision from two choices to four S3 storage classes. And we will replace the yes-or-no reward with a reward measured in actual money, where wrongly sending a needed scan to archive is punished far more heavily than simply paying a little extra for faster storage.

3.4 Paper 9 | Kaapana: A Comprehensive Open-Source Platform for Integrating AI in Medical Imaging Research Environments

U. Akunal, M. Bujotzek, S. Denner et al., German Cancer Research Centre (DKFZ), arXiv:2512.09644, 2025. Code at github.com/kaapana/kaapana.

In one line: *A complete, free, working hospital imaging platform that already has a PACS, object storage, a DICOM search index and AI pipelines, and yet has no intelligence at all about where each scan should be kept.*

What the authors did:

The authors built and released a full open-source platform for medical imaging research. It brings data ingestion, patient group selection, processing workflows and result viewing into one place. It is built on Kubernetes with Helm for deployment, Apache Airflow to run workflows as step-by-step job graphs, MinIO for S3-compatible object

storage, OpenSearch to index and search DICOM metadata, and Keycloak for login and access control. It includes a built-in PACS that accepts DICOM, a dataset browser, ready-made AI pipelines such as TotalSegmentator and nnU-Net, and image viewers. Its design principle is to send the algorithm to the data rather than move patient data out of the hospital.

Data they used:

This is infrastructure, so there is no dataset and no accuracy number. The evidence is real deployment. Kaapana is in use in the RACOON national radiology network, the CCE-DART consortium, the German Cancer Consortium (DKTK) and the NeuroRad project.

Advantages:

- It is proof that a platform like this really works in hospitals, not only in a paper.
- Its storage layer, MinIO, speaks the same S3 language as Amazon S3. This means the tier logic we write can be plugged in directly with very little change.
- OpenSearch already indexes every DICOM tag of every scan. That index is precisely the source of features our model needs, and it is already built and running.
- Airflow gives us a natural place to run our tier recommendation as a scheduled background job.
- It is completely free and open source, which suits a student project with no budget.
- The send-the-algorithm-to-the-data design fits privacy rules and would let our model be trained across several hospitals without moving patient data.

Limitations:

- Storage is completely flat. Everything sits in MinIO buckets at one single quality of service. There is no concept of a hot tier or a cold tier anywhere.
- There is no cost awareness at all. The platform never asks what the storage is costing.
- There is no automatic lifecycle. Nothing ever moves a scan to cheaper storage as it gets older.
- The AI pipelines are all diagnostic. They segment organs and find disease. AI is never used to run the infrastructure itself.
- The paper reports no measurement of storage efficiency, and states no limitations section of its own.

RESEARCH GAP

Kaapana is the clearest example of the gap in this entire study, because it already owns every single ingredient our project needs and connects none of them. It has a searchable index of every DICOM tag. It has S3-compatible object storage. It has a workflow engine that can run jobs on a schedule. It has deep learning pipelines running over the images. And yet every scan, from a routine X-ray to an urgent trauma CT, is dropped into the same bucket and left there for ever. The DICOM metadata that OpenSearch has already indexed is exactly what a tier prediction model would need, and it is used only to help a researcher search. The link from what the system already knows to where the scan should be stored simply does not exist.

How we can improve on this:

This gives our project a real home instead of a simulation. We will build our tier recommender as an Airflow job inside Kaapana. It will read the DICOM features it needs from the OpenSearch index that is already there, run our deep learning model, and write the resulting storage class decision out to S3 lifecycle rules. That turns Kaapana from a flat store into a cost-aware, tiered archive, and it means our result is something a hospital could actually install rather than only a set of graphs.

3.5 Paper 10 | Project Imaging-X: A Survey of 1000+ Open-Access Medical Imaging Datasets for Foundation Model Development

Z. Deng and 126 co-authors, *arXiv:2603.27460*, 2026. Portal at github.com/uni-medical/Project-Imaging-X.

In one line: *A catalogue of more than a thousand free medical imaging datasets, which solves half our data problem and proves that the other half cannot be solved off the shelf at all.*

What the authors did:

A very large team catalogued over 1,000 publicly available medical imaging datasets and organised them by imaging type, clinical task, body region, kind of annotation, known limitations, and how easily they can be combined with others. They also propose a method they call the metadata-driven fusion paradigm, which is a way of joining several small datasets that share a modality or a task into one larger and more useful dataset. They released an interactive search portal and a single structured table listing everything.

Data they used:

The datasets themselves are the data. There is no experiment and no model. The output is the catalogue, the portal and the public repository.

Advantages:

- It directly solves our requirement for a free and openly available dataset. Instead of searching one dataset at a time, we can filter a single table.
- It records the known limitations of each dataset, so we can choose one honestly and state its weaknesses in our report.
- The fusion idea lets us combine several small datasets into something large enough to train on.
- It is from 2026, so it is completely current.
- The portal and repository are public and free.

Limitations:

- It is a catalogue, not a method. There is no algorithm, no experiment and no result to compare against.
- Every dataset in it is a diagnostic dataset. They are all built to train models that find disease.
- Not a single dataset records storage or access behaviour.
- The authors describe the whole landscape as fragmented, modest in scale, and unevenly spread across body parts and imaging types.
- Many public datasets have been converted to PNG or JPEG, which destroys the DICOM header that our features depend on.
- The authors note that curation is limited by the need for clinical expertise and by strict privacy rules.

RESEARCH GAP

This paper is the strongest evidence we have that our data problem is real. More than a thousand medical imaging datasets are freely available, and not one of them records who opened which scan and when. That record lives only inside hospital PACS audit logs, it is protected patient information, and it is never released publicly. So the very thing our model must learn to predict, namely whether a scan will be needed again, cannot be downloaded from anywhere. It has to be derived. On top of this, many public datasets have been stripped down to plain PNG or JPEG images, which throws away the DICOM header that half of our input features come from.

How we can improve on this:

We will use the catalogue to pick a dataset that still keeps its DICOM headers and that has several scans per patient over time. From those timestamps we will build a proxy label. The reasoning is simple: when a patient comes back for a follow-up scan, the radiologist almost always pulls up the earlier scan to compare. So a later scan for the same patient is good evidence that the earlier one was retrieved. We will state clearly in the report that this is a proxy and not a true access log, explain that it will miss retrievals that happen without a new scan, and test how sensitive our results are to that assumption. Being open about this is stronger than pretending we have real access logs.

4. Overall Research Gap

Putting all five papers together, this is what I found:

- **Paper 6 (Harmonia)** can place files and move them between tiers very well, and it proves the overhead is tiny, but it judges a file only by how it has behaved and never by what it contains.
- **Paper 7 (Sibyl)** proves that learning beats hand-written rules for storage placement, but it uses only six inputs and every one of them is about system behaviour, so a brand new scan cannot be placed intelligently at all.
- **Paper 8 (Cold-RL)** shows how to train such a system safely from recorded logs instead of live experiments, but it only chooses between keep and delete, and it treats every mistake as equally bad.
- **Paper 9 (Kaapana)** already has the DICOM index, the object storage, the workflow engine and the AI pipelines that our project needs, and connects none of them to a storage decision. Every scan sits in the same bucket for ever.
- **Paper 10 (Imaging-X)** proves that over a thousand free medical imaging datasets exist and that not one of them records which scan was opened when, so the label our model needs must be derived rather than downloaded.

So the gap is this. The storage research community has built accurate, extremely fast, learning based tier systems, but it feeds them only block-level behaviour and has never once tested them on medical data. The medical imaging community has built rich deep learning models and complete hospital platforms, but uses them only to help the doctor and never to manage the storage underneath. Nobody has joined the two. No existing work uses deep learning on the medical image itself, together with its DICOM details and its past access record, to predict whether a scan will be needed again and to place it in the correct Amazon S3 storage class, with a penalty that treats a doctor waiting for an archived scan as a far worse mistake than a slightly higher bill. That is what our project will do.

5. Literature Survey Table

Paper	Method	Dataset	Advantages	Limitations	Research Gap
P6 - Nadig et al. (2025), Harmonia, arXiv:2503.20507, SAFARI Group, ETH Zurich	Two small reinforcement learning agents that coordinate with each other. A placement agent decides which device a new file is written to; a migration agent decides when an existing file should be moved. Both are kept small so a decision is faster than the device itself.	25 real workloads: SYSTOR17 enterprise traces (LUN0-4), RocksDB (ssd-00 to 04), YCSB (B-F), MLPerf Storage (ResNet50, CosmoFlow), 3 LLM traces, and 5 unseen Meta production traces (Baleen). Real hardware: Intel Optane P4800X, Intel D3-S4510, Seagate HDD, ADATA SU630, in 2, 3 and 4 device setups.	Handles placement and migration together, which our project needs. 29.3% lower latency on a speed-focused setup and 44.8% on a cost-focused setup; 136.7% more IOPS. Scales to 4 devices, matching the four S3 classes. Only 240 ns per decision and 206 KIB memory. Tested on unseen workloads.	No medical data anywhere; workloads are office, database and ML jobs. Agents see only behaviour numbers, never file content. "Cost" means cheaper hardware, not a cloud bill; no transfer or retrieval charges. Settings tuned once by hand. No legal retention rule.	A follow-up cancer CT and a routine chest X-ray with the same size, age and open count are treated identically, so the important scan can be sent to deep archive and take hours to return. Cost is never measured as an actual monthly bill, and there is no way to keep a scan for the years hospital rules require.
P7 - Bera, Nadig & Mutlu (2026), arXiv:2603.14583, ETH Zurich	Overview paper covering three learning based methods: Pythia (RL prefetcher), Hermes (perceptron cache predictor) and Sibyl (RL data placement for mixed storage). Sibyl describes each page using exactly six numbers and learns which device to place it on.	No new experiments. Sibyl was tested on real machines using 14 storage traces from Microsoft Research Cambridge enterprise servers. Its six inputs are request size, read or write, access interval, access count, free space on the fast device, and current placement.	Proves the core idea works: Sibyl beat hand-written rules by 19.9-45.9% and earlier learning methods by 34.1-68.8%. Tested on real hardware in the OS storage layer, not only simulation. States its six inputs openly. Honest that Sibyl cannot do migration, which led to Harmonia.	All six inputs describe behaviour, none describe content. MSRC traces are ordinary enterprise servers with no medical data. Cold start problem: a new file has no access history. Works between devices in one machine, not cloud storage classes or cloud pricing. Overview paper, so no new experiment to repeat.	The six-input list is the gap in one line. A hospital already knows the modality, body part, patient age, urgency and report text at the moment a scan is created, all free in the DICOM header, and all of it is ignored. The cold start problem makes it worse: exactly when the placement decision must be made, the system knows the least about the file.
P8 - Gupta & Bhayani (2025), Cold-RL, arXiv:2508.12485, code on GitHub	A dueling Deep Q-Network replaces the fixed least-recently-used rule in the NGINX web server. When space is needed it scores the K least recently used objects on age, size, number of opens and time to live, then picks which to remove, all within a microsecond budget.	No live system was experimented on. Real NGINX access logs were replayed through a cache simulator and the model learned offline from that replay. Reward: a kept file that is opened again before expiry earns a point. Cache sizes 25 MB, 100 MB and 400 MB.	Offline training from recorded logs is the only method a hospital would permit, and this proves it works. Its four inputs map almost one to one onto scan attributes we already have. Hit rate rose 14.36% to 35.38% at 25 MB and 75.30% to 86.75% at 100 MB. Under 2% CPU overhead. Honest that at 400 MB it gave no benefit. Code is public.	A web cache, not a storage archive: it decides what to delete, not which of four classes to use. Only two outcomes. Reward is a simple yes or no. No money is measured anywhere. Benefit shrinks as cache grows. No medical data and no images.	The decision here is two-way (keep or delete) and every mistake costs the same: one slow re-fetch that nobody notices. Our decision is four-way, and a wrong move to Glacier Deep Archive means a doctor waits twelve hours for a scan. A yes-or-no reward cannot express that difference at all.
P9 - Akunal et al. (2025), Kaapana, arXiv:2512.09644, DKFZ Germany	A full open-source medical imaging platform. Kubernetes and Helm for deployment, Apache Airflow for workflows, MinIO for S3-compatible object storage, OpenSearch to index DICOM metadata, Keycloak for login. Includes a built-in PACS, dataset browser, TotalSegmentator and nnU-Net pipelines, and image viewers.	Infrastructure, so no dataset and no accuracy figure. Evidence is real deployment: the RACOON national radiology network, CCE-DART consortium, German Cancer Consortium (DKTK) and NeuroRad project.	Proof that such a platform works in real hospitals. MinIO speaks S3, so our tier logic plugs in directly. OpenSearch already indexes every DICOM tag, which is exactly our feature source. Airflow gives a natural home for our job. Completely free and open source. Send-the-algorithm-to-the-data design suits privacy rules and multi-	Storage is completely flat; everything sits in one class of MinIO bucket with no hot or cold tier. No cost awareness at all. No automatic lifecycle, so nothing ever moves as a scan ages. AI pipelines are purely diagnostic, never used to run the infrastructure. No storage efficiency measurement, and no limitations section.	It owns every ingredient our project needs and connects none of them. A searchable DICOM index, S3 storage, a scheduler and deep learning pipelines all exist, yet a routine X-ray and an urgent trauma CT go into the same

			hospital training.		bucket for ever. The metadata OpenSearch has already indexed is precisely what a tier model needs, and it is used only for search.
P10 - Deng et al. (2026), Project Imaging-X, arXiv:2603.27460	A survey cataloguing over 1,000 open-access medical imaging datasets, indexed by modality, clinical task, body region, annotation type, known limitations and how easily they combine. Proposes a metadata-driven fusion paradigm to merge small datasets into larger ones.	The datasets themselves are the data; there is no experiment and no model. Outputs are an interactive discovery portal, a single unified structured table, and a public GitHub repository.	Directly solves our need for a free and open dataset: one filterable table instead of searching one at a time. Records each dataset's known limitations so we can choose honestly. The fusion idea lets us combine small sets into something trainable. Published 2026, so completely current. Portal and repository are free.	A catalogue, not a method: no algorithm, experiment or result to compare against. Every dataset is diagnostic. None record storage or access behaviour. The landscape is described as fragmented, modest in scale and unevenly spread. Many datasets are converted to PNG or JPEG, destroying the DICOM header. Curation limited by clinical expertise and privacy rules.	Over a thousand free medical imaging datasets exist and not one records who opened which scan and when. That record lives only in hospital PACS audit logs, is protected patient information, and is never released. So the label our model must predict cannot be downloaded and has to be derived from follow-up timestamps instead.

Note: All five papers above are openly available at no cost. Direct links - P6: arxiv.org/abs/2503.20507 | P7: arxiv.org/abs/2603.14583 | P8: arxiv.org/abs/2508.12485 | P9: arxiv.org/abs/2512.09644 | P10: arxiv.org/abs/2603.27460

AI-Based Storage Tier Recommendation for Medical Image Repositories using Deep Learning Analytics

Research Gap Analysis and Literature Survey (Papers 11 to 15)

Course Instructor: Dr. Priya V | Submission: 30 July 2026

Name: Mehul Anand

Reg No:24BIT0185

1. What This Project Is About

Hospitals store a very large number of scans such as CT, MRI and X-ray. Keeping all of them on fast storage is expensive. Moving all of them to cheap storage is also a problem, because cheap storage can take hours to give a file back, and a doctor cannot wait that long.

Amazon S3 offers different storage classes. S3 Standard is fast but costly. Glacier Deep Archive is very cheap but slow. The aim of this project is to use deep learning to decide which storage class each scan should go into.

For this part of the literature survey I studied papers 11 to 15. For each paper I have written what the authors did, what data they used, what is good about it, what is missing, and what gap is left that our project can fill.

2. The Five Papers I Studied

#	Title	Journal	Year
P11	Machine learning-driven automatic storage space recommendation for object-based cloud storage system	Complex & Intelligent Systems (Springer)	2022
P12	Secure and fault tolerant cloud based framework for medical image storage and retrieval	Scientific Reports (Nature)	2025
P13	PARADIM: A Platform to Support Research at the Interface of Data Science and Medical Imaging	J. Imaging Informatics in Medicine (Springer)	2025
P14	A Guideline for Open-Source Tools to Make Medical Imaging Data Ready for AI Applications	J. Imaging Informatics in Medicine (Springer)	2024
P15	BVFLEMR: Federated Learning and Blockchain for a Cloud-Based Medical Records Recommendation System	Journal of Cloud Computing (Springer)	2022

3. Research Gap Analysis (Paper by Paper)

3.1 Paper 11 — Machine learning-driven automatic storage space recommendation for object-based cloud storage system

A. S. Mondal, A. Mukhopadhyay and S. Chattopadhyay, *Complex & Intelligent Systems*, vol. 8, pp. 489-505, 2022.

In one line: This paper uses machine learning to pick a storage location automatically, but it decides from server readings only and never looks at the file itself.

What the authors did: When data arrives at their cloud storage system, called RSoS, a monitoring tool named Ganglia watches the server while the data is being written and records 18 readings such as CPU, memory, input/output and load. A feature selection step keeps only the useful readings, and a classifier then predicts whether the incoming data is a sensor reading, a document or a file. Based on that prediction, the data is sent to the matching database: Cassandra, MongoDB or HDFS. They tried three feature selection methods (Fisher score, F-score and LII21) with three classifiers (SVM, K-NN and Neural Network), so nine combinations in total. LII21 with K-NN worked best.

Data they used: No public dataset was used. They made their own by running the storage system on one workstation and recording what the server did. Each row has the 18 server readings plus a label saying which of the three data types it was. The three types were kept in an equal 1:1:1 ratio. Health data was used as the example workload, including sensor streams such as pulse, temperature, SpO2, airflow and ECG, along with documents and files.

Advantages:

- It proves the main idea our project depends on: a machine learning model can decide by itself where a piece of data should be stored.
- Before this, a person had to edit a schema file by hand every time a new data type arrived. That manual step is removed completely.
- Sending each data type to the database that suits it reduces query time and helps save energy in the data centre.
- The features are server readings, not the data itself, so patient data is never opened. That is good for privacy.
- The nine-way comparison is useful for us. It tells us which feature selector and classifier to start with instead of guessing.

Limitations:

- Training is done offline. The authors say themselves that if the model learns something wrong, there is no way to correct it while the system is running.
- It knows only three classes: sensor, document and file. Medical images in DICOM format are not handled at all.
- Everything ran on one local workstation. It was never tested on a real cloud such as AWS, so we do not know how it behaves under real load.
- The dataset is small and made by the authors themselves, so nobody else can repeat the experiment or compare against it.
- It chooses a database, not a storage class. Cost is never calculated anywhere in the paper.
- Even though healthcare data is used, there is no encryption and no access control.
- It was published in 2022, which is outside the 2023 to 2026 range preferred in the guidelines. I have included it because it is the only paper that treats storage placement itself as a learning problem.

RESEARCH GAP

The model here decides where data goes by watching the server, not by looking at the data. For text and sensor readings that may be acceptable, but for medical images it is not. Two CT scans written to the same server at the same moment will produce almost the same CPU, memory and load readings, so this model

would treat them identically, even if one is a routine normal scan and the other is one a doctor will open again next week. The decision is also taken once, offline, and can never be corrected afterwards. And the choice is between databases, never between storage classes with different prices, so the question our project asks, which S3 class should this scan go into, is never asked here.

How we can improve on this: We keep the core idea and change what the model looks at. Instead of server readings, our model will look at the scan itself using deep learning, together with DICOM details such as scan type and body part, and the hospital's record of how often similar scans were opened again. The output will be an Amazon S3 storage class such as Standard, Standard-IA, Glacier Instant Retrieval or Deep Archive, instead of a database name, so the decision is tied directly to cost. We will also fix the offline problem the authors admit to, by checking the decision again through S3 Lifecycle rules so a scan can be moved later if it starts being opened more often.

3.2 Paper 12 — Secure and fault tolerant cloud based framework for medical image storage and retrieval in a distributed environment

A. Amaithi Rajan, V. Vetrivel, M. Ajitesh and R. Praveen Kumar, Scientific Reports, vol. 15, art. 32965, 2025.

In one line: This paper protects and retrieves medical images across several clouds, but it keeps every scan in the same place at the same cost.

What the authors did: They build a framework called SFMedIR with four parts. First, federated learning: several hospitals train a shared model, based on a ConvNeXt network, that turns each image into a short code, without ever sending their images to one another. Second, the images are encrypted using a quantum-chaos method that is meant to stay safe even against future quantum computers. Third, a shadow scheme splits the stored data across one master cloud and several slave clouds, so nothing is lost if one cloud fails. Fourth, the whole system runs inside Docker containers and was deployed on Amazon EC2.

Data they used: Three public medical image datasets were used: Alzheimer Brain MRI with 5,120 training, 1,280 testing and 1,000 retrieval images; Brain Tumour MRI with 7,023 images across four classes; and Kidney CT with 10,000 training, 2,466 testing and 1,000 retrieval images. Attacks were simulated using the standard PGD method. Retrieval accuracy was about 90 percent overall, 5 to 10 percent better than earlier models, 60 to 70 percent better on attacked queries, and still above 75 percent while under attack.

Advantages:

- It shows that a deep learning model can turn a medical scan into a short code that describes what is inside it. Our project needs exactly this kind of image understanding.
- Federated learning means the hospital never has to send raw patient images anywhere, so our project can train the same way without breaking privacy rules.
- It handles security, fault tolerance and retrieval accuracy together, where most papers handle only one of the three.
- It was actually deployed on Amazon EC2, so the design is not purely theoretical.
- There is a formal security proof as well as experiments, and three separate datasets were used, so the results are not tied to one collection of images.

Limitations:

- The experiments assume a perfect world: honest servers, secure channels, and a stable network with almost no packet loss. A real hospital network is not like this.
- It was only a small Docker simulation on EC2, not a real deployment across regions over a long period.
- Encryption, federated training and the shadow scheme are all heavy work, but the paper never says what this costs in money or in extra waiting time for the user.
- Every image is treated the same way. There is no idea of a frequently used scan and a rarely used one, so there is no cheap storage anywhere in the design.
- Copying data across master and slave clouds actually multiplies the storage bill, and that bill is never reported.
- It builds its own security instead of using ready-made cloud services such as AWS KMS and IAM, which makes it harder to certify against hospital rules.
- No radiologist checked whether the images it retrieves are actually good enough to make a diagnosis.

RESEARCH GAP

This paper makes storage safe and reliable but never makes it cheap. Every scan is encrypted, split and copied across clouds in exactly the same way, whether it is opened every day or never again. The short code the model produces describes what the image contains, and that is precisely the information needed to guess whether a scan will be wanted again. But it is used only for searching. It is never sent back to the storage side to decide where the scan should live. So the paper builds the exact tool our project needs and then uses it for a different purpose.

How we can improve on this: We will take the image code idea and point it at a different question. The same code this paper uses for searching can also be used to predict how likely a scan is to be opened again, and therefore which S3 class it belongs in. Cheap protection can come from AWS itself, using S3 with KMS encryption and Cross-Region Replication instead of a custom shadow scheme, and only the important scans need the expensive replication. Most importantly, we will report the monthly bill and the time taken to get a scan back, which this paper leaves out completely.

3.3 Paper 13 — PARADIM: A Platform to Support Research at the Interface of Data Science and Medical Imaging

Y. Lemarechal et al., Journal of Imaging Informatics in Medicine, vol. 39, pp. 1783-1795, 2025.

In one line: This is a complete working platform for hospital scans, but all the scans sit on the same storage and nothing ever decides to move them.

What the authors did: They built a platform called PARADIM out of free open-source parts, and it is in real use today. Karnak removes patient identity from the scans while they are still inside the hospital. Kheops organises the scans into collections and controls who can see them. Orthanc is the actual store where the DICOM images are kept. Keycloak handles logins with multi-factor authentication. The OHIF viewer lets researchers look at and label images. An in-house Job-Manager running on Kubernetes launches heavy jobs such as automatic segmentation, dose calculation and AI

model testing. Everything stays in DICOM format from start to finish, and every step is recorded so results can be traced and repeated.

Data they used: This is not an experiment with a dataset. The platform holds real medical image collections belonging to the institution, including CT, MRI, radiotherapy plans and dose maps, and is used by around ten research projects that have ethics approval.

Advantages:

- It shows a complete working pipeline: bring the scan in, remove patient identity, store it, label it, train on it and publish the result. Our project has to fit inside a pipeline like this one.
- Keeping everything in DICOM means details such as body part, scan type and acquisition settings are never lost. Our model needs exactly those details as input features.
- Security and governance are handled properly, with logins, multi-factor authentication, permissions and a full record of who did what.
- It is built from mature free components, so licensing cost is low and any part can be replaced.
- It directly attacks the reproducibility problem caused by datasets of unknown origin, which is a real weakness in medical AI.

Limitations:

- It runs on the institution's own machines. When a big job arrives there is no way to borrow extra capacity, and when the machines are idle the money is still spent.
- The platform moves data around but never thinks about it. There is no model that decides where a scan should be kept or when it can be moved to cheaper storage.
- All scans sit on the same storage. There are no tiers, no archive and no cost figures anywhere in the paper.
- The authors note that outside analysis tools still need the data to be downloaded locally, which weakens the security boundary they worked hard to build.
- It handles DICOM images only. Patient records, lab results and doctors' notes cannot be stored in it.
- Running it needs real engineering skill in Kubernetes, OpenShift and Orthanc plugins, which a small hospital or a student team will not have.
- Labelling is still partly manual. Web-based contouring is described as work that is still going on.

RESEARCH GAP

PARADIM is the best example of how hospital images should be organised, and it is exactly the kind of place where our project would sit. But it has no cost thinking at all. Every scan, whether it was viewed fifty times this month or last opened four years ago, is kept on the same storage at the same price. The platform already records everything needed to make a better decision, namely who opened which scan, when, and how often. That record is used only for tracing the research. It is never fed back to decide where the scan should live, and there is no automatic archiving, so old scans stay on expensive storage for ever.

How we can improve on this: We will follow the same structure but place it on AWS so storage can be tiered and cost can be controlled: Amazon S3 for the images, AWS Lambda for de-identification, Amazon Cognito in place of Keycloak, and Amazon SageMaker for training. Then we add the missing piece, which is a deep learning model that reads the scan and the access history PARADIM already

collects, and predicts which S3 class the scan belongs in. S3 Lifecycle rules then carry out the move automatically, so no person has to do it.

3.4 Paper 14 — A Guideline for Open-Source Tools to Make Medical Imaging Data Ready for AI Applications: A SIIM Survey

S. Vahdati et al., Journal of Imaging Informatics in Medicine, vol. 37, pp. 2015-2024, 2024.

In one line: This paper lists the free tools that get medical images ready for AI, but it never asks where those images should be stored or what that costs.

What the authors did: This is a survey, not a system. A questionnaire was sent to 500 members of the Society of Imaging Informatics in Medicine, asking which free tool they use for preparing medical images and why. 54 replies came back from 26 medical informatics centres. After removing repeats, vague answers and tools that were not publicly available, 28 tools were left. Each tool was then described by what it can do, including removing patient identity, viewing images, working in the cloud, converting formats, normalising, labelling, annotating, segmenting, storing and supporting federated learning, and the tools were grouped by which stage of preparation they belong to.

Data they used: The survey itself is the data: 54 responses from 26 centres, reduced to 28 tools such as 3D Slicer, the RSNA Anonymizer, CTP, DIANA, dcm2niix, ITK-SNAP, MITK, MONAI Label, MIDRC, Niffler and CVAT. No medical images were analysed at any point.

Advantages:

- It is genuinely useful for us. It saves weeks of searching when we have to choose a tool to remove patient identity or to convert DICOM files before feeding them to our model.
- It covers the whole preparation pipeline rather than one step, so we can see how the stages connect to each other.
- The answers come from people working in 26 real informatics centres, not from advertising material.
- It marks which tools work in the cloud and which support federated learning, which matters directly for a cloud project like ours.
- It warns about specific traps, such as tools that can only produce one label when several labels are needed.

Limitations:

- Nothing is measured. There are no numbers for speed, memory or accuracy, so we still cannot say which tool is best, only which ones are popular.
- Only 54 of 500 people replied, about 11 percent, and all from one society, so the list may not reflect world practice.
- Tools change quickly, so the list goes out of date. The authors say this themselves.
- Paid tools and in-house tools were left out, even though many hospitals depend on them.
- The tools are listed one by one. There is no design showing how to join them into an automatic pipeline.
- Cost, security and legal compliance of running these tools at scale are not examined.

- Working in the cloud is recorded as a simple yes or no. Which cloud, and at what cost, is never discussed.

RESEARCH GAP

This paper stops at the exact moment our project starts. It explains carefully how to get an image ready for a model, and then says nothing about where the image goes afterwards. Storage appears only as a tick-box feature of a tool, never as a decision with a price attached. None of the 28 tools looks at how often a scan is opened, none of them chooses a storage class, and none of them can move a file to cheaper storage later. Worse, the information our model needs is actually created during preparation, such as scan type, body part and image quality, but it is thrown away once the image has been converted.

How we can improve on this: We will use the tools this survey recommends, but chain them into an automatic pipeline instead of running them by hand: a scan lands in Amazon S3, an event starts AWS Step Functions, and AWS Lambda runs de-identification and format conversion. At the same time we will keep the details these tools produce, such as scan type, body part, file size and image quality, and pass them into our tier model as features instead of discarding them. We will also record the time taken and the cost per scan, which is the number this survey never gives.

3.5 Paper 15 — BVFLEMR: An Integrated Federated Learning and Blockchain Technology for Cloud-Based Medical Records Recommendation System

T. Hai, J. Zhou, S. R. Srividhya, S. K. Jain, P. Young and S. Agrawal, Journal of Cloud Computing, vol. 11, art. 22, 2022.

In one line: This paper decides what to do with patient records using machine learning, but it works on text records and keeps everything stored the same expensive way.

What the authors did: The system has two parts. In the first, electronic health records are written into a Hyperledger Fabric blockchain that sits on a cloud server, so every change to a record is permanently recorded and cannot be secretly altered. In the second, vertical federated learning collects information from several of these blockchains, and a LightGBM model together with an N-Gram language model and sentiment analysis recommends a suitable treatment for the patient.

Data they used: An electronic health record dataset was used together with a free sentiment and emotion library. The paper never says which dataset it is, how large it is, or where it came from. The data was fed in step by step at 20, 40, 60, 80 and 100 percent. Average precision was 0.953, recall 0.932 and F1 score 0.950. LightGBM is said to be around 20 times faster than similar gradient boosting models.

Advantages:

- It shows that decisions about healthcare data can be treated as a machine learning problem, which is the same assumption our project makes.
- LightGBM is fast and light on memory, so it is realistic for a student project. Our tier model can use it as a baseline before trying anything heavier.
- Every change to a record is tracked, which gives a natural audit trail for hospital rules.
- Precision, recall and F1 score are all reported, not only accuracy, so the results can be judged properly.
- Federated learning is used, so hospitals never have to hand over their raw records.

Limitations:

- The dataset is not named, not sized and not sourced. Nobody can repeat the work or compare against it.
- The results get worse as more data is added. Recall falls from 0.952 at 20 percent to 0.920 at 100 percent, and the paper never explains why.
- Blockchain is expensive and slow, but the paper never measures write speed, delay or storage cost. These are the main reasons hospitals hesitate to use it.
- Only text and tabular records are handled. Medical images, which take up most of the storage in a hospital, are not covered at all.
- Everything is stored in the same way. There is no cheap storage, no archive and no idea that some records are needed less often than others.
- The treatment recommendations were never checked by a doctor, and the model cannot explain why it recommended anything.
- It was published in 2022, which is outside the 2023 to 2026 range preferred in the guidelines. I have included it because it is the closest work to the idea of learning where healthcare data should be kept.

RESEARCH GAP

This paper puts intelligence in front of storage, which is the right idea, but the storage behind it has only one setting. A blockchain keeps every record permanently and at full cost, so a record nobody has opened in five years costs exactly as much as one being used today. The decision is also based on the text of the record, and no image is ever examined. For an image archive, where a single CT study can be hundreds of megabytes, this design would be extremely expensive. Nothing here predicts whether a record will be needed again, and that is the one prediction that decides the bill.

How we can improve on this: We will keep the useful part, which is treating the storage decision as a learning problem and using LightGBM as a fast baseline, but we will change what it decides. Our model works on one scan at a time, using features taken from the image itself plus its access history, and it chooses an Amazon S3 storage class instead of storing everything permanently. Only a small hash of each scan needs to be kept in a tamper-proof ledger such as Amazon QLDB, while the scan itself sits in the S3 class the model chose. We will also report the cost and the time taken to get a scan back, which this paper never does.

4. Overall Research Gap

Putting all five papers together, this is what I found:

- Paper 11 shows that a model can choose a storage location on its own, but it decides from server readings and never looks inside the file, and it picks a database rather than a priced storage class.
- Paper 12 builds a deep learning model that describes what is inside a scan, but uses that description only for searching, and stores every scan the same expensive way.
- Paper 13 gives a complete, working platform for hospital images, but keeps every scan on one type of storage and never decides to move anything.
- Paper 14 explains how to get images ready for AI, but treats storage as a tick box and never mentions cost or storage class.

- Paper 15 does treat a healthcare data decision as a learning problem, but works on text records, keeps everything permanently, and never measures what that costs.

So the gap is the same one seen from five directions. Every one of these papers either understands the image but ignores the storage bill, or manages the storage but never looks at the image. No existing work reads the medical image itself with deep learning, combines that with the record of how often the scan was opened before, and uses the result to predict whether the scan will be needed again and place it in the correct Amazon S3 storage class. That is exactly what our project will do.

5. Literature Survey Table

Paper	Method	Dataset	Advantages	Limitations	Research Gap
P11 — Mondal et al. (2022), Complex & Intelligent Systems, Springer	Ganglia records 18 server readings (CPU, memory, I/O, load) for each write. Feature selection (Fisher, F-score, LII21) plus a classifier (SVM, K-NN, Neural Network) predicts if the data is sensor, document or file, then sends it to Cassandra, MongoDB or HDFS. LII21 with K-NN was best.	Own data from running RSoS on one workstation. 18 server readings per write plus a label, three classes in 1:1:1 ratio. Health workload of sensor streams (pulse, temperature, SpO2, airflow, ECG), documents and files.	Proves a model can choose a storage location by itself. Removes a manual schema edit. Matching data type to database cuts query time and saves energy. Patient data is never opened. Useful nine-way comparison of selectors and classifiers.	Training is offline and cannot be corrected at run time. Only three classes, no DICOM. Tested on one workstation, not a real cloud. Small self-made dataset. Picks a database, not a storage class. No cost calculation, encryption or access control.	Two CT scans written at the same moment give almost the same server readings, so an important scan and a routine one are treated alike. The decision is taken once, offline, and is between databases rather than priced storage classes.
P12 — Amaithi Rajan et al. (2025), Scientific Reports, Nature	SFMedIR: federated learning with a ConvNeXt network turns each image into a short code without sharing images; quantum-chaos encryption protects the images; a shadow scheme spreads data across master and slave clouds for fault tolerance. Run in Docker on Amazon EC2.	Alzheimer Brain MRI (5,120/1,280/1,000), Brain Tumour MRI (7,023 images), Kidney CT (10,000/2,466/1,000). PGD attacks simulated. About 90% retrieval accuracy, 5-10% above earlier models, 60-70% better on attacked queries.	Shows a model can turn a scan into a short code describing its content. Federated learning keeps raw images inside the hospital. Handles security, fault tolerance and accuracy together. Deployed on Amazon EC2. Formal security proof plus three datasets.	Assumes honest servers and a near-perfect network. Only a small Docker simulation on EC2. Cost and extra delay never measured. Every image treated the same, so no cheap storage. Copying across clouds multiplies the bill. Custom security instead of AWS KMS and IAM.	Storage is made safe and reliable but never cheap. The image code describes what is inside a scan, exactly what is needed to guess if it will be wanted again, but it is used only for searching and never sent back to the storage side.
P13 — Lemarechal et al. (2025), J. Imaging Informatics in Medicine, Springer	PARADIM platform built from open-source parts: Karnak removes patient identity, Kheops manages collections and access, Orthanc stores the DICOM images, Keycloak handles login with MFA, OHIF is used for labelling, and an in-house Job-Manager on Kubernetes runs heavy jobs.	Not an experiment. Holds real institutional collections of CT, MRI, radiotherapy plans and dose maps, used by about ten ethics-approved research projects.	A complete working pipeline from ingest to publication. Staying in DICOM keeps scan type, body part and settings, which our model needs as features. Proper login, MFA, permissions and full traceability. Built from free components.	Runs on the institution's own machines, so no extra capacity when needed and wasted money when idle. Moves data but never reasons about it. All scans on one storage: no tiers, no archive, no cost figures. DICOM only. Needs heavy DevOps skill.	Every scan costs the same whether it was viewed fifty times this month or last opened four years ago. The platform records who opened which scan and how often, but uses it only for tracing research, never to decide where the scan should live.
P14 — Vahdati et al. (2024), J. Imaging Informatics in Medicine, Springer	A survey, not a system. Questionnaire sent to 500 SIIM members, 54 replies from 26 centres, reduced to 28 open-source tools. Each tool described by what it can do (de-identify, view, cloud, convert, normalise, label, annotate, segment, store, federated learning) and grouped by preparation stage.	The survey itself: 54 responses from 26 medical informatics centres, reduced to 28 tools such as 3D Slicer, RSNA Anonymizer, CTP, DIANA, dcm2niix, ITK-SNAP, MITK, MONAI Label, MIDRC, Niffler and CVAT.	Saves weeks when choosing a de-identification or conversion tool. Covers the whole preparation pipeline. Answers come from 26 real informatics centres. Marks which tools work in the cloud and which support federated learning.	Nothing measured, so no evidence of which tool is best. Only about 11% replied, all from one society. List dates quickly. Paid and in-house tools excluded. Tools listed separately with no automatic pipeline. Cloud support is a yes/no flag.	It explains how to prepare an image and then says nothing about where it goes. No tool tracks how often a scan is opened, chooses a storage class, or moves a file later. Details created during preparation are thrown away instead of reused.
P15 — Hai et al. (2022), Journal of Cloud Computing, Springer	BVFLEMR: health records are written into a Hyperledger Fabric blockchain on a cloud server so every change is permanent and traceable. Vertical federated learning then collects from several blockchains and a LightGBM model with an N-Gram language model and sentiment analysis recommends treatment.	An unnamed EHR dataset plus a free sentiment and emotion library. Size, source and feature count are not reported. Fed in at 20/40/60/80/100%. Precision 0.953, recall 0.932, F1 0.950. LightGBM said to be about 20 times faster.	Treats a healthcare data decision as a machine learning problem, the same assumption our project makes. LightGBM is fast and light, usable as our baseline. Every record change is auditable. Reports precision, recall and F1, not just accuracy.	Dataset unidentified, so not reproducible. Results get worse as data grows (recall 0.952 to 0.920) with no explanation. Blockchain speed, delay and storage cost never measured. Text records only, no images. Everything stored the same way with no cheap tier.	Intelligence sits in front of storage that has only one setting. A record untouched for five years costs the same as one in daily use. The decision reads text, never an image, and nothing predicts whether the data will be needed again.

AI-Based Storage Tier Recommendation for Medical Image Repositories using Deep Learning Analytics

Part D | Abstract, Objectives, Novelty, Architecture, Dataset and AWS Services

Course Instructor: Dr. Priya V | Submission: 30 July 2026

Pratyush Chandrasekhar (24BIT0226) | Subhrojyoti Das (24BIT0194) | Mehul Anand (24BIT0185)

Repository: github.com/SUBHRO71/SmartStorageTierRecommenderForMedicalSector_Cloud_Project_2026

D1. Abstract

Hospital imaging archives now grow by terabytes every year, and the cost of keeping that data is one of the largest recurring line items in a radiology department's IT budget. Storing every scan on high-performance cloud storage is financially unsustainable, but moving everything to cheap archival storage is clinically unsafe: retrieving a study from Amazon S3 Glacier Deep Archive can take up to twelve hours, and a radiologist comparing a follow-up scan against a prior cannot wait that long.

Existing solutions fail from opposite directions. Cloud storage tiering, including AWS's own S3 Intelligent-Tiering and the learning-based systems in the recent literature, decides where a file belongs using only block-level behaviour such as object size, age and access frequency. It is completely blind to what the file contains. Medical deep learning, meanwhile, has become very good at reading scans, but its output is used only to assist the radiologist and is never fed back to the storage layer. Two identical-looking files, one a routine normal chest X-ray and the other an active oncology follow-up, are treated identically today.

This project closes that gap. A deep learning model reads each scan and, together with its DICOM metadata and access history, predicts the probability that the study will be retrieved again. An explicit cost rule then converts that probability into one of four Amazon S3 storage classes by minimising expected total cost, deliberately weighting a wrongly archived scan far more heavily than a marginally higher storage bill. The decision is written as an S3 object tag and enforced by a tag-filtered Lifecycle rule, so the model sets policy while AWS performs the mechanism.

The system is deployed entirely serverless on Amazon S3, AWS Lambda, Amazon DynamoDB, Amazon API Gateway, Amazon Cognito, Amazon CloudFront, Amazon CloudWatch, Amazon SNS, AWS IAM and AWS Budgets, so idle cost is zero and the whole system fits inside always-free AWS allowances.

Word count: approximately 290.

D2. Project Objectives

Each objective below is stated so that it can be measured and either met or missed. Targets are first-pass goals and will be reported honestly against in Phase-II, including where they are not met.

- 1. Build a deep learning tier recommender.** Train a model that predicts, for each individual scan, the probability that it will be retrieved again, using image-derived features together with DICOM metadata and access history. Target: ROC-AUC of at least 0.75 on a patient-disjoint test split, with a calibration curve reported alongside, since the cost rule consumes the probability directly.
- 2. Convert probability into a storage class using an explicit, auditable cost rule.** Select from S3 Standard, Standard-IA, Glacier Flexible Retrieval and Glacier Deep Archive by minimising expected total cost. The rule must account for storage, request, transition and retrieval charges, minimum storage durations, the 128 KB minimum billable object size and the 40 KB per-object Glacier metadata overhead.

3. **Reduce projected storage cost measurably.** Achieve at least a 40 percent reduction in projected monthly storage cost against an all-S3-Standard baseline, and outperform both a conventional age-based lifecycle rule and S3 Intelligent-Tiering on the same 112,120-record dataset.
4. **Keep clinically unsafe placements rare.** Hold the wrongly-archived rate, defined as the proportion of scans that were subsequently needed but had been placed in Glacier or Deep Archive, below 5 percent. This metric is reported next to the cost saving in every result, never separately.
5. **Deploy a working serverless system on AWS within the free tier.** Demonstrate an end-to-end path from scan upload through automated tier decision to a real S3 Lifecycle transition, using no EC2 or RDS, with total AWS spend below one US dollar.
6. **Deliver an authenticated dashboard that explains its decisions.** Provide a Cognito-authenticated web dashboard showing tier distribution, projected cost, and for any individual scan the predicted probability and the reason it was assigned its storage class.

D3. Novelty Summary

Across the fifteen papers surveyed by this team, no existing work uses the content of a medical image, together with its metadata and access history, to decide which cloud storage class that image should occupy. The novelty of this project is therefore not an incremental improvement to an existing method; it is the connection of two mature fields that have not previously been joined.

New feature | content-aware storage tiering

Every tiering system in the surveyed literature judges a file by behaviour alone. Sibyl, for example, describes each object with exactly six numbers: request size, request type, access interval, access count, free capacity and current placement. Not one of them describes what the data is. A hospital already knows the modality, body part, patient age, study description and reported findings at the moment a scan is created, all of it free in the DICOM header. This project is the first to feed that information, plus a learned representation of the image itself, into the placement decision.

Better algorithm | asymmetric, two-stage decision

Rather than a four-class classifier, which would treat every misclassification as equally bad, the system separates a learned probability from an explicit cost rule. This makes each decision explainable, allows the clinical penalty to be re-tuned and every result regenerated without retraining, and encodes in inspectable code the fact that stranding a needed scan in Deep Archive is a far worse error than paying slightly more for fast storage.

Better architecture | policy separated from mechanism

The model does not move data. It writes an S3 object tag, and a tag-filtered Lifecycle rule performs the transition. AWS handles retries and auditing, the decision is visible in the console, and a failure in the model can never leave an object half-migrated.

Better AWS integration | a cost model that is actually correct

The closest prior work, Khan et al. (2024), explicitly omitted inter-tier transfer fees and early-deletion penalties, and our own gap analysis identified this as a limitation. This project models the full charge structure, including the 128 KB minimum billable object size and the 40 KB per-object Glacier metadata overhead, which are invisible at the headline per-GB rate but materially change the economics of archiving medical images. Reported savings are therefore conservative and defensible rather than optimistic.

Better security and automation

Amazon Cognito authenticates users and API Gateway validates the resulting tokens, while AWS IAM grants each component only the permissions it needs. The pipeline is fully event-driven: a scan upload triggers the decision automatically, with no scheduled job and no human in the loop, and Amazon SNS raises an alert whenever a scan is retrieved from archive so that mistakes surface immediately rather than being discovered by a waiting clinician.

Better scalability

Being entirely serverless, the system scales from a single scan to an entire hospital archive without provisioning, and costs nothing when idle. Inference runs in Lambda in milliseconds because the expensive image-embedding step is precomputed offline rather than repeated per request.

D4. Proposed Architecture

Two diagrams are provided, as required. Figure 1 shows how the AWS services interact, covering data flow, storage, processing, authentication, notifications and monitoring. Figure 2 shows the overall project workflow from dataset through to evaluation. Both appear on the following landscape pages.

Both are generated by `architecture/make_diagrams.py` in the repository, so they can be regenerated whenever the design changes rather than being redrawn by hand. The Mermaid sources in `architecture/README.md` remain the master reference, and the reasoning behind each design choice, including the alternatives rejected, is recorded as eight architecture decision records in `architecture/decisions.md`.

Diagram 1 | AWS Cloud Architecture

AWS Cloud Architecture

Smart Storage Tier Recommender for Medical Image Repositories | BCSE355L Project Phase-I | Region: us-east-1

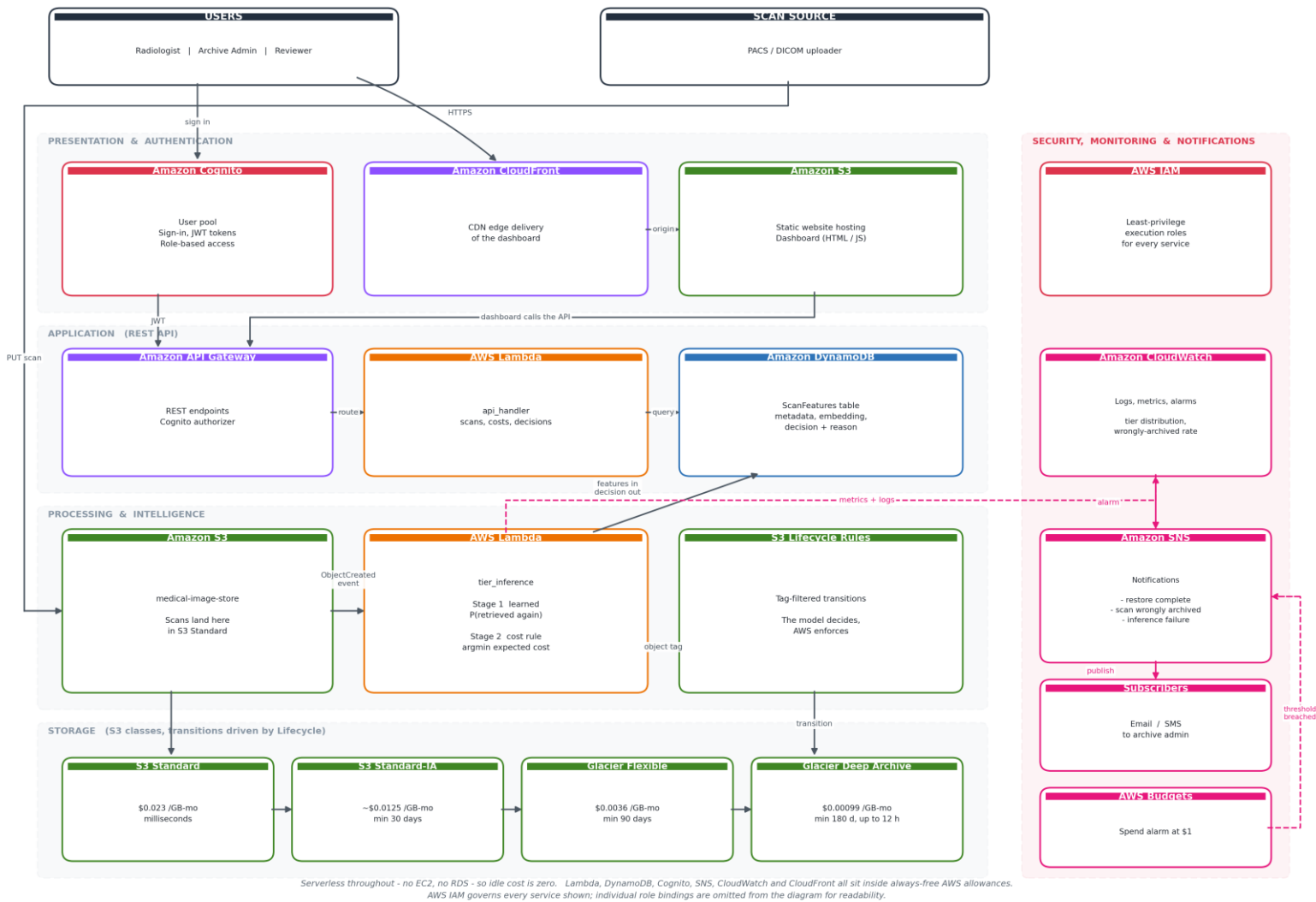


Figure 1. AWS Cloud Architecture. Four layers - presentation and authentication, application, processing and intelligence, and storage - with a cross-cutting security, monitoring and notification column on the right.

Diagram 2 | Complete System Architecture

Complete System Architecture

End-to-end workflow: offline training, online tier decision, feedback loop and cost evaluation



Figure 2. Complete System Architecture. The offline training pipeline, the per-scan online decision, the feedback loop from retrieval events, and the offline cost evaluation against three baselines.

D4.1 How the AWS services interact (Figure 1)

The architecture is arranged in four horizontal layers with a cross-cutting security, monitoring and notification column on the right.

- **Authentication.** A radiologist or archive administrator signs in through an Amazon Cognito user pool, which issues a JWT. Amazon API Gateway is configured with a Cognito authorizer, so every API call is validated before it reaches any Lambda function.
- **Presentation.** The dashboard is a static site held in Amazon S3 and delivered through Amazon CloudFront. It holds no credentials and never contacts AWS services directly; it calls the REST API only.
- **Application.** Amazon API Gateway routes requests to an `api_handler` Lambda, which reads and writes an Amazon DynamoDB table holding each scan's metadata, image embedding, predicted class and the reason for that decision.
- **Data flow and processing.** A new scan is uploaded to the `medical-image-store` S3 bucket and lands in S3 Standard. The resulting `ObjectCreated` event triggers the `tier_inference` Lambda, which reads the scan's features from DynamoDB, runs the Stage 1 model to obtain a retrieval probability, applies the Stage 2 cost rule, and writes the chosen class back as an S3 object tag.
- **Storage.** A tag-filtered S3 Lifecycle rule performs the actual transition between S3 Standard, Standard-IA, Glacier Flexible Retrieval and Glacier Deep Archive. Lifecycle evaluates once daily, so transitions are not instantaneous.
- **Monitoring.** Amazon CloudWatch collects logs and custom metrics, notably the distribution of scans across classes and the wrongly-archived rate, and raises alarms on threshold breaches.
- **Notifications.** Amazon SNS publishes to email and SMS subscribers when an archive restore completes, when a scan is retrieved from cold storage and therefore logged as wrongly archived, and when inference fails. AWS Budgets publishes through the same topic if spend crosses one dollar.
- **Security.** AWS IAM provides least-privilege execution roles for every component. The inference function can tag objects in one bucket and write to one table, and can do nothing else.

D4.2 Overall system workflow (Figure 2)

The workflow runs in four stages. Stage 1 is offline and free: the ChestX-ray14 dataset is parsed, the retrieval label is derived, MobileNetV3 produces a 128-dimensional embedding per image on a free Colab GPU, and the Stage 1 probability model is trained. The resulting model artifact is deployed to Lambda and the embeddings are loaded into DynamoDB.

Stage 2 is the online per-scan decision already described. Stage 3 is the feedback loop: when a doctor requests a scan, a fast-tier hit is served in milliseconds, while an archive miss incurs a restore job of several hours, is logged as wrongly archived and raises an SNS alert. Either way the access counters in DynamoDB are updated and feed the next decision for that scan.

Stage 4 is evaluation, performed offline across all 112,120 records so that no AWS cost is incurred. The full cost model is applied to our policy and to three baselines, and the monthly bill is always reported together with the wrongly-archived rate.

Note on scale

The mechanism is demonstrated live on approximately 1,000 real images, which fits inside the 5 GB always-free S3 allowance, while cost is projected across all 112,120 records using AWS's published price list. This is stated explicitly rather than presented as a full-scale measurement. The same approach was taken by Khan et al. (2024), who used Google Cloud prices on simulated data; this project improves on it by including the transition and early-deletion costs that paper omitted.

D5. Dataset Details

Attribute	Detail
Dataset Name	NIH ChestX-ray14 (also published as ChestX-ray8)
Source	National Institutes of Health Clinical Center, USA. Released with Wang et al., "ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases", CVPR 2017.
URL	https://nihcc.app.box.com/v/ChestXray-NIHCC Mirrors: Kaggle (nih-chest-xrays/data) and Academic Torrents
Size	Approximately 45 GB in total, distributed as 12 compressed archives. Average image size approximately 400 KB. A working subset of roughly 1,000 images (about 400 MB) is used for the live AWS demonstration.
Number of Records	112,120 frontal-view chest radiographs from 30,805 unique patients. Mean of approximately 3.6 images per patient.
Number of Features	Raw metadata: 11 columns in Data_Entry_2017.csv (Image Index, Finding Labels, Follow-up #, Patient ID, Patient Age, Patient Gender, View Position, Original Image Width and Height, and Original Image Pixel Spacing x and y). Engineered feature vector: 140 dimensions, comprising a 128-dimensional MobileNetV3 image embedding plus approximately 12 metadata and derived features (patient age, gender, view position, follow-up index, prior study count, encoded finding labels, object size in bytes).
Data Type	Images: PNG, 1024 x 1024 pixels, 8-bit greyscale. Metadata: CSV, mixed categorical and numeric. Derived label: binary.
License	Publicly released by the NIH Clinical Center for research and educational use. No registration, credentialing or data use agreement is required. Attribution to Wang et al. (CVPR 2017) is required in any publication. The images are de-identified; no protected health information is present.
Purpose of using the dataset	Two purposes. First, to derive the retrieval-likelihood label that the Stage 1 model learns, using the Follow-up # column. Second, to supply the image content and metadata features that make this project content-aware rather than behaviour-only. It was selected specifically because it requires no clearance of any kind, which suits an undergraduate team with no institutional data access.
Preprocessing required	1. Parse Data_Entry_2017.csv and group rows by Patient ID. 2. Derive the binary label: a scan is labelled 1 if its Follow-up # is below the maximum for that patient, indicating a later study exists and the earlier one was very likely retrieved for comparison. 3. Split by Patient ID, never by image, so that scans of the same patient cannot appear in both training and test sets. 4. Resize to 224 x 224 and normalise for the embedding network. 5. Extract the 128-dimensional embedding for every image. 6. Encode categorical metadata; derive prior-study counts and approximate intervals from Patient Age deltas. 7. Address class imbalance and record the fixed random seed for reproducibility.

Known limitations of this dataset, stated openly

- No timestamps. ChestX-ray14 records follow-up ordering but not dates, so intervals are approximated from Patient Age deltas and are coarse.
- The label is a proxy, not ground truth. It cannot capture retrievals that occur without a subsequent scan, such as multidisciplinary team meetings, second opinions or research requests. A sensitivity analysis on this assumption is planned.
- Images are distributed as PNG rather than DICOM, so genuine DICOM header fields are simulated from the available metadata columns.
- Chest radiography only. Findings will not necessarily generalise to CT or MRI, whose objects are far larger and whose retrieval patterns differ.

Why no public dataset can supply the true label

No publicly available dataset records who retrieved which scan and when. Those records exist only inside hospital PACS audit trails in IHE ATNA format, are protected health information, and are never released. The training label therefore cannot be downloaded and must be derived. There is published precedent for exactly this proxy: Yayan et al., Scientific Reports, December 2025, on metadata-derived proxies of 90-day follow-up in the same 112,120 ChestX-ray14 radiographs.

D6. AWS Services Planning

AWS Service	Purpose in this project	Free-tier position	Owner
Amazon S3	Stores the medical images; also hosts the static dashboard. The four storage classes are what the model predicts.	5 GB Standard, 20,000 GET, 2,000 PUT per month	Mehul, Pratyush
AWS Lambda	tier_inference runs the two-stage decision on upload; api_handler serves the REST API.	1M requests and 400,000 GB-seconds per month, always free	Mehul, Subhrojyoti
Amazon DynamoDB	Stores per-scan metadata, the image embedding, access history and the recorded decision with its reason.	25 GB and 25 RCU/WCU, always free	Subhrojyoti
Amazon API Gateway	Exposes the REST API to the dashboard, with a Cognito authorizer validating every request.	Consumes credits beyond the free allowance	Subhrojyoti
Amazon Cognito	Authenticates radiologists and archive administrators; issues JWTs and enforces role-based access.	Generous monthly active user allowance	Subhrojyoti
Amazon CloudFront	Delivers the dashboard from edge locations.	Always-free monthly allowance	Pratyush
Amazon CloudWatch	Logs, custom metrics such as tier distribution and wrongly-archived rate, dashboards and alarms.	10 custom metrics and 5 GB of log ingestion free	Mehul, Pratyush
Amazon SNS	Notifies on archive restore completion, wrongly archived scans, inference failures and budget threshold breaches.	1M publishes and 1,000 email notifications free	Mehul
AWS IAM	Least-privilege execution roles and policies for every component.	Free	Subhrojyoti
AWS Budgets	Spend alarm set at one dollar, configured before any other resource is created.	Free	All members

Services deliberately not used

Amazon EC2 and Amazon RDS are excluded by design. AWS replaced the Free Tier for new accounts on 15 July 2025; accounts created after that date receive credits on a six-month clock rather than twelve months of free EC2. Lambda, DynamoDB, Cognito, SNS and CloudFront retain always-free monthly allowances that neither consume credits nor expire. A fully serverless design therefore has zero idle cost, and no forgotten instance can drain the budget. Amazon SageMaker is also excluded: model training runs free on Google Colab, and inference is small enough to run inside Lambda.

D7. Implementation Status and Repository

Deliverable	Status
Literature survey, 15 papers across the team	Complete
Individual research gap analyses	Complete, Parts A to C of this document
Abstract, objectives, novelty	Complete
AWS Cloud Architecture diagram	Complete, Figure 1
Complete System Architecture diagram	Complete, Figure 2
Dataset selection and documentation	Complete, NIH ChestX-ray14
AWS services plan	Complete
GitHub repository, branches and pull requests	Active
Preprocessing and label derivation	Not started, Phase-II
Stage 1 model training	Not started, Phase-II
Cost model and baseline comparison	Not started, Phase-II
AWS deployment	Not started, Phase-II

Repository

github.com/SUBHRO71/SmartStorageTierRecommenderForMedicalSector_Cloud_Project_2026

Branching follows main, develop and one feature branch per member. The architecture design and this documentation were merged through pull request #1. Architecture decisions, including the alternatives that were rejected and why, are recorded as eight ADRs in `architecture/decisions.md`.

Immediate next steps for Phase-II

7. Confirm the AWS account creation date, which determines the applicable free-tier regime.
8. Create the AWS Budgets alarm at one dollar before provisioning any other resource.
9. Download the ChestX-ray14 subset, derive the label and build patient-disjoint splits.
10. Train and calibrate the Stage 1 probability model.
11. Implement the full cost model and compare against the three baselines.
12. Deploy the serverless stack and demonstrate a real Lifecycle transition.